Sequel2SQL : An Agentic Framework for SQL Error Correction

Aravindh Manavalan, Vijay Balaji S, Akshay Ravi, Smeet Dedhia & Jay Sanghavi
Sponsored by Dhruv Relwani

The Problem

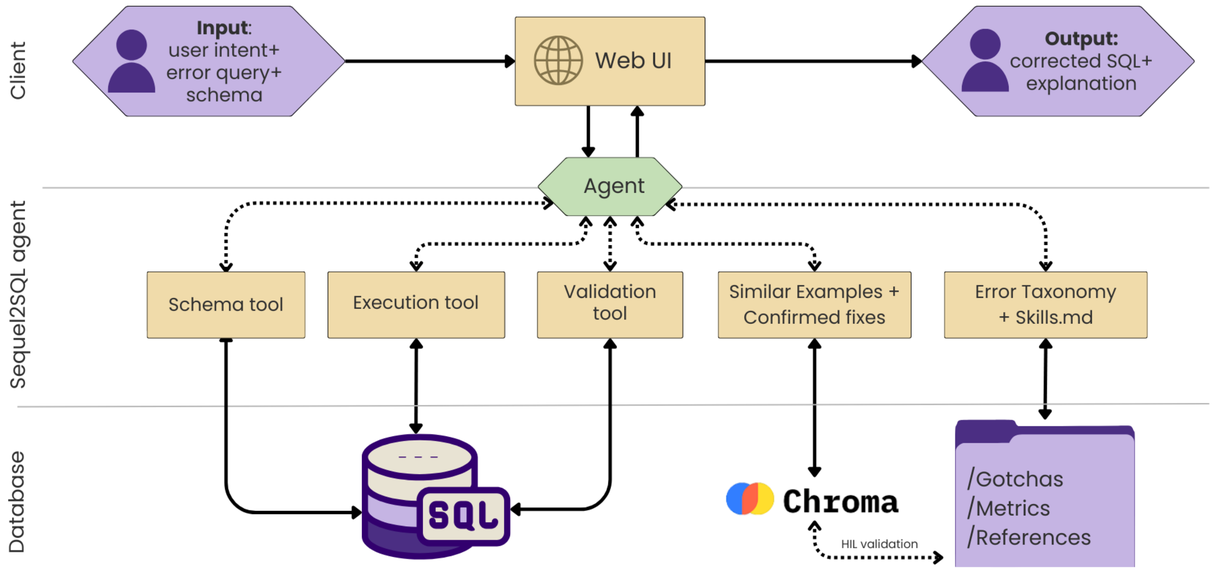
Many AI tools excel at generating **SQL (NL2SQL),** however they still struggle to reliably fix broken queries in real-world database environments. In practice, data engineers and analysts spend a significant portion of their time debugging issues such as syntax errors, incorrect joins, hallucinated columns, aggregation mistakes, and schema mismatches. Generic large language models often lack the database context, validation mechanisms, and reliability required to correct SQL queries and often, fail to address the problem. We built an intelligent system focused on **PostgreSQL** error correction, and query optimization.

Our Objectives

- Design a modular system that can work with various models in a plug and play architecture.
- Improve SQL query correction beyond a standalone LLM response.
- Reduce hallucinations such as nonexistent tables, columns, and incorrect joins.
- Ground the model with database schema, validation loops, and added context before generating a fix.
- Allow for iterative learning to equip the system for constantly evolving databases, and to improve overall efficiency.

System Overview

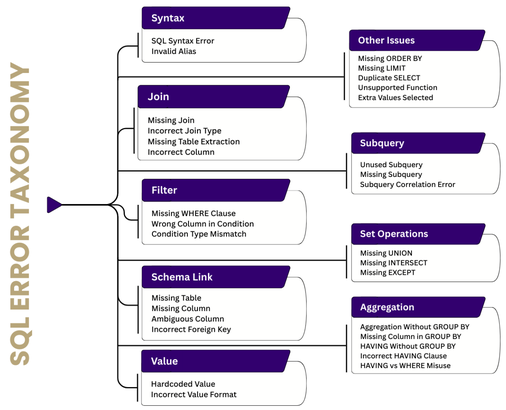
Built on **PydanticAI**, our system combines agent workflows, tools, validation, skills, logging, and a web UI in one platform. A **ChromaDB RAG layer** retrieves similar past fixes, while **live database execution** lets the agent test queries against real errors and refine them iteratively. Guardrails ensure only safe and valid queries are executed.



Sequel2Sql Agent System Architecture

Novelty & Contributions

We introduced a novel multi-strategy validation pipeline that combines static AST analysis with live PostgreSQL EXPLAIN feedback to catch errors that standard parsers miss. Finally, we mapped over **40 PostgreSQL error codes** to an error taxonomy, giving the agent a playbook for fixing specific issues.



Errors Classified by Sequel2Sql Validation pipeline

Our system features a diversity-aware retrieval engine that uses **Maximal Marginal Relevance (MMR)** to find structurally distinct query examples, preventing redundant context. We also built a feedback loop, which allows the system to verify repairs and store them in **ChromaDB** and in **skills.md** for the final human-validated semantic model**,** effectively letting it "learn" and specialize to the database without any retraining.

The entire system is **agentic** in such a way that the model can choose and pick the most relevant tool required to solve a particular query, improving efficiency and problem solving.

Evaluation / Benchmarks

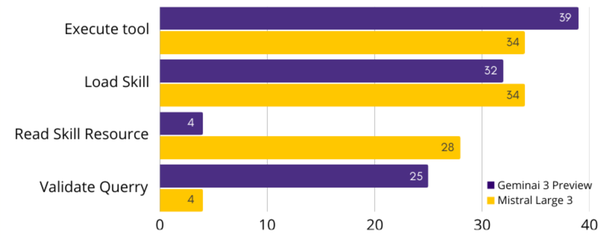
To test our system, we used the **BIRD-CRITIC-1.0-PG** benchmark. We ran our agent against a subset of the 530 real-world erroneous queries across 15+ different schemas, executing every corrected query in a secure Docker container. The fix was only counted if the output matched the ground truth exactly and passed all assertions, which is scored on comparing the result sets, SQL keyword usage, and query execution plan.

The results showed that our agentic framework significantly elevates model capabilities, though base reasoning still matters.

Gemini 3 Flash Preview jumped from **42% to 48%** accuracy, proving it could effectively use validation and retrieval to catch complex errors.

Mistral Large 3, however, actually saw a dip to 25%. This suggests that smaller models can struggle with context rot and don't always use tool calls as effectively.

Model	BIRD-CRITIC-PG (%)
Human Performance	76.7
Gemini 3 Flash Preview	42.1
Gemini 3 Flash with tools	**48.0**
Mistral Large 3	32.6
Mistral Large 3 with tools	25.5



Tool Usage Across 30 SQL queries

Future Work

Redesign Sequel2SQL as a Model Context Protocol server that would allow any MCP-compatible client to connect to any database or warehouse and allow continual learning on the DB. Correction skills stored directly in the connected warehouse would keep the system's learned knowledge transparent, auditable, and shared with the data team.